

Task 1: CPU-Based Heat Propagation

1.1 Objective:

To simulate directional heat propagation inside a horizontally cylindrical radiator using finite differences on the CPU. The propagation occurs only along each row (horizontal direction) with periodic boundary conditions (wrap-around), mimicking the behavior of water flowing through pipes.

1.2 Implementation Summary:

Language: C++ with `std::vector<std::vector<float>>` for 2D matrix representation.

Matrix Allocation: Two `n × m` float matrices are dynamically allocated based on user-specified `-n` and `-m` values (default 32).

Boundary Conditions:

- Column `0` is initialized as

$$matrix[i][0] = 0.98 \cdot \frac{(i+1)^2}{n^2}$$

- Other columns use:

$$matrix[i][j] = matrix[i][0] \cdot \frac{(m-j)^2}{m^2}$$

- Heat Propagation:** Applied for `p` steps (default 10), using the directional stencil:

$$new[i][j] = \frac{1.60 \cdot A_{j-2} + 1.55 \cdot A_{j-1} + 1.00 \cdot A_j + 0.60 \cdot A_{j+1} + 0.25 \cdot A_{j+2}}{5.0}$$

- Wrap-Around:** Periodic indexing is handled via modulo arithmetic for left/right neighbors.
- Row Average:** If the `-a` flag is given, compute and display the average temperature per row.

1.3 Verification & Testing

Tested on:

- Small sizes: `5x5`, `8x10` with `p=10`
- Output values are as expected and stable
- All command-line options are parsed and functional (`-n`, `-m`, `-p`, `-a`)
- Wrap-around correctly handled even at boundaries (verified via manual inspection)

Task 2: GPU-Based Heat Propagation using CUDA

2.1 Objective

To accelerate the same directional heat propagation model on the GPU using CUDA, while maintaining output consistency with the CPU version and reporting speedup and accuracy.

2.2 Implementation Summary

Language: CUDA C++

Kernel Design:

- 1D or 2D grid/block setup (configurable)
- Each thread computes one cell in the matrix
- Propagation and wrap-around logic mirrors CPU version
- ** ow Average Calculation:**
- Implemented as a separate kernel for modularity and independent timing

Timing & Comparison:

- Uses `cudaEvent_t` for:
 - GPU memory allocation
 - Host→Device transfer
 - Kernel execution
 - Row average computation
 - Device→Host transfer
- Controlled via `-t` flag

Error Analysis:

- GPU result compared element-wise to CPU
- Reports any mismatch $>1e-4$
- Prints max absolute error

2.3 Verification & Testing

Kernel tested for:

- Small grids (`5x5` , `8x10`) with exact matching results
- Medium grids (`512x512`) with negligible error (e.g. `max error ~1e-5`)

Functional options: `-c` to skip CPU, `-a` , `-t`

Wrap-around logic behaves correctly on GPU (visually and numerically verified)

Mismatch detection tested with altered GPU result to ensure logic correctness

2.4 Summary

- Both CPU and GPU implementations match the mathematical model and agree in results.
- The GPU implementation is modular and prepares the foundation for Task 3 performance benchmarking.
- Precision is preserved within $1e-4$ error tolerance.
- All required command-line flags and logic have been successfully implemented.
- Code has been modularized (`.cc` , `.cu` , `.h`), and Git commits clearly track development steps.

Task 3 – GPU Block Size Performance Test Summary

n this task, we evaluated the performance of the GPU-based heat propagation solver using different thread block sizes. The simulation was performed on a large grid of size `15360 × 15360` for `p = 1000` propagation steps. The goal was to determine how thread block size affects GPU execution time, and compare it with the CPU performance.

3.1 Configuration:

Matrix size: `15360 × 15360`

Time steps: `p = 1000`

CPU: OpenMP-parallelized version

GPU: CUDA kernel with row-major access and shared memory

Timing measured with `std::chrono` for CPU and `cudaEvent` for GPU

Precision: **single precision (float)**

3.2 Test Results:

Matrix size	Block size	Propagation Step(p)	CPU time(ms)	GPU time(ms)	Speedup(cpu/gpu)	MaxError
15360 × 15360	128	1000	921558.605467	2067.79	445	2.1e-5
15360 × 15360	64	1000	921558.605467	2076.81	444	2.1e-5
15360 × 15360	32	1000	921558.605467	2050.33	449	2.1e-5
15360 × 15360	16	1000	921558.605467	2042	451	2.1e-5
15360 × 15360	8	1000	921558.605467	2041.98	451	2.1e-5

On my test machine, the GPU executed the full simulation in **2050 ms**, which is faster than the reference 5s reported in the spec.
This is likely due to either newer hardware, reduced memory overhead, or kernel efficiency.
The output was verified against the CPU with a max difference of `xxxex-5`.

3.3 Observations:

All block sizes yielded **very similar GPU performance**, ranging from 2041 ms to 2076 ms.
The **CPU time was consistently over 921,000 ms (~15 minutes)**, indicating high computational cost on the CPU.
Speedup factors were in the range of **444× to 451×**, demonstrating excellent GPU acceleration.
The **maximum absolute difference** between CPU and GPU results was **2.1e-5**, well below the acceptable tolerance `1e-4`, confirming correctness.

3.4 Conclusion:

The GPU implementation shows massive acceleration over the CPU version for large-scale problems, with minimal precision loss. Thread block size has a small but measurable impact on GPU time. A block size of **16 or 8** gave slightly better performance in our case.

Task 4 Report: Double-Precision GPU Simulation

4.1 Objective

The goal of Task 4 was to extend the GPU heat propagation simulation to support double-precision floating-point computation. This involves modifying the CUDA implementation to support `double` instead of `float`, and ensuring correct execution and accuracy comparison.

4.2 Implementation

Code Refactoring with Templates

To support both float and double types without duplicating code, we implemented the GPU simulation logic using C++ function templates. The `run_gpu_simulation<T>` function is now templated on the precision type `T`, and its behavior is selected at runtime via the `-d` flag.

CUDA Kernel Support for Double

Two templated CUDA kernels were implemented:

- `gpu_heat_step<T>`
- `gpu_compute_row_averages<T>`

Special handling was added for atomic additions in double precision. Since `atomicAdd` for `double` is only supported on devices with compute capability ≥ 6.0 , we used conditional compilation:

```
#if __CUDA_ARCH__ >= 600
    atomicAdd(...); // double
#else
    // fallback logic (not implemented in this assignment)
#endif
```

Command-Line Option

A new runtime flag `-d` was introduced to switch to double-precision mode. When provided, the program calls the templated simulation with `T = double`, otherwise it defaults to `float`.

4.3 Results & Observations

1. Execution

- The GPU simulation executed successfully in both float and double precision for small matrix sizes.
- For large matrices (e.g., 15360 x 15360), both float and double sometimes caused segmentation faults when trying to print large result matrices. This issue was isolated to post-processing and output.

2. Output Example

Float mode (partial):

```
GPU Execution Time (float): 1656.027466 ms
Segmentation fault (core dumped)
```

Double mode (partial):

```
GPU Execution Time (double): 928.560303 ms
Segmentation fault (core dumped)
```

This indicates the kernel executed and timed successfully, but accessing the output matrix afterward caused a memory fault, likely due to uninitialized or oversized vectors.

3. Accuracy

- The computed average temperatures using float and double precision were similar, with negligible differences at small matrix sizes.
- Due to the segmentation fault at large sizes, automated max error computation between float and double could not be completed.

4.4 Challenges

- Lack of native support for `atomicAdd(double)` on older devices made the implementation dependent on GPU compute capability.
- Managing template specializations and function dispatch cleanly without duplicating code.
- Segmentation faults due to potentially uninitialized vectors or overly large output accesses.

4.5 Conclusion

Task 4 successfully extended the GPU simulation with double-precision support using templated kernels and runtime selection via a `-d` flag. The double-precision version runs correctly for small to medium grid sizes. Further debugging is required to ensure stable output printing and comparison at very large matrix sizes.